

The arguments

The callable takes two arguments—*environ* and *start_response()*. The first one, *environ*, is a dictionary that holds the environment, which is basically a *CGI* environment with a few WSGI-specific keys added. The canonical way to parse a WSGI *environ* is to import the *cgi* module and use its *FieldStorage* class, as follows:

```
import cgi

# We'll make a copy of the environ and strip out its query
# string, because apparently it's bad for FieldStorage

post_env = environ.copy()
post_env["QUERY_STRING"] = ""

# And now we'll parse it

post = cgi.FieldStorage(
    fp = environ["wsgi.input"],
    environ = post_env,
    keep_blank_values = True
)
```

The more interesting argument is the second one—*start_response()* which is a callable. *Start_response()* is used to set the status code and send out the headers. But why can't that be done as part of returning the content? Well, it's an ugly hack, but a very clever one that is immensely useful and no one has suggested a better way yet.

One of the aims of WSGI is to make the entire thing very loosely coupled, infinitely pluggable and layerable. With this kind of flexibility, one might think the spec would require a huge API. But the Python developers have managed to pack in this extreme flexibility with just the one call interface.

The *start_response()* call tells your server what response your app wants to send to the browser (or client). But the next trick is, the status (and headers) are sent only after the app has returned, or has yielded at least one element to send to the client. This gives ample room to set up a *202 Accepted* response

and go on processing a particularly expensive operation, and change it to a *200 OK* or a *400 series* error code, depending on the outcome of the operation just before returning the data. But where this hack really shines is in WSGI middleware.

Middleware have some of the most interesting types of WSGI apps. These WSGI apps call other WSGI apps. Middleware can add elements to the request, or remove elements that are meant for the middleware to consume. As the response flows back from your app to the middleware and then on to the client, the middleware can analyse and change the response. Middleware solves pretty generic issues, like HTTP authentication, URL-based routing, sub domain-based routing, XSRF protection, and the like.

Middleware can be stacked infinitely. Because they are WSGI apps and look like WSGI compliant servers to the upstream apps, as many WSGI middleware as are needed can be chained together.

This is where *start_response()* really shines. The first middleware calls *start_response()* with *202 Accepted* and calls the next one in the chain to handle the request. Suddenly, one middleware pretty high up in the chain decides to throw a *500 Internal Server Error*. As this response travels back down the middleware chain, another middleware picks this up, turns it into a *503 Service Unavailable*, and returns a pretty looking *503 page* with the webmaster's e-mail.

The return

WSGI's return is even more interesting. An app (the callable) needs to return an iterable. The idea is that the app can return one chunk of data as and when it becomes available. The specification actually states that the headers are first sent out when the first non-empty string is yielded, so an error state can be achieved pretty late into the handling cycle. The only catch is that no non-empty strings should have been returned before changing a *200 OK* into a *500 Internal Server Error*.

There's an excellent article on the Internet about the benefits of WSGI—the link is provided in the resource box at [1]. It should definitely be read before attempting to write your first WSGI application. It gives you a feel of what the developers were trying to achieve, and what you should and shouldn't do with the spec.



GET ROBUST, SCALABLE & COST-EFFECTIVE OPEN SOURCE SOLUTIONS

- TechnoMail - Enterprise Email Server Anti SPAM, Anti Virus, Email Content Filtering
- Firewall, Internet Access Control Content Filtering, Site Blocking
- Bandwidth Management System
- Managed Email Hosting Solutions

TechnoInfotech®

1, Vikas Permisses, 11 Bank Street,
Fort Mumbai, India-400 001,
Mobile: 09167399917.
Email: info@technoinfotech.com